

MID-STATE TECHNICAL COLLEGE • IT PROGRAMS

Hypervisor Installation and Architecture

Unit 2 • Virtualization and Software Defined Data Centers (10-150-130)



Learning Outcomes

By the end of this unit, you will be able to:



Plan a hypervisor deployment

Verify hardware compatibility and size CPU, memory, storage, and network before installing.



Install Proxmox VE

Boot the installer, configure ZFS RAID1, set the management network, and verify a working host.



Explain the architecture

Describe how Debian, KVM, and LXC combine into a Type 1 virtualization platform.



Apply capacity planning

Choose oversubscription ratios and identify the metrics that reveal resource contention.

LESSON 1

Planning and System Requirements



Plan Before You Boot the Installer

Four decisions are effectively permanent once the host carries production workloads:

- **CPU features** — virtualization extensions must be enabled in UEFI before install, not after.
- **Storage layout** — the RAID level and filesystem chosen in the installer cannot be changed in place.
- **Management network** — the address set at install becomes the cluster identity of the node.
- **Hostname and domain** — renaming a Proxmox node after it joins a cluster is a rebuild, not an edit.



In your POD

Your **HP DL20 Gen 10** hosts are already racked and cabled. Planning here means reading what is in front of you — firmware level, disk count, NIC count — and recording it before you install.

CPU Virtualization Extensions

Hardware-assisted virtualization is what makes a Type 1 hypervisor practical. Verify all four:

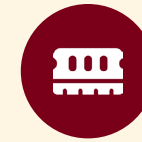
Feature	Intel	AMD	What it does
CPU virtualization	VT-x	AMD-V	Runs guest code directly on the core
I/O virtualization	VT-d	AMD-Vi	Passes devices through to a guest
Nested paging	EPT	NPT	Translates guest memory in hardware
Trusted execution	TXT	SVM / SEV	Attestation and memory encryption

Nested paging is the one students forget; without it, memory-heavy guests lose double-digit performance.

Sizing Host Memory

Guest memory is not the whole number — the hypervisor takes its share first.

- Reserve **2–4 GB** for the Proxmox VE host itself, plus overhead per running guest.
- ZFS adds an **ARC cache** that will consume free memory by design — budget for it deliberately.
- **Allocated** memory is what you promised a guest; **consumed** memory is what it actually touches.
- Plan against consumed memory, but never let allocated memory outrun what failure can absorb.



ZFS is memory-hungry

A rough working figure is **1 GB of RAM per 1 TB** of ZFS storage, before deduplication. Deduplication multiplies that and is almost never worth enabling in a teaching lab.

Storage Layout and Redundancy

The boot and guest storage layout is chosen once, in the installer:

Layout	Usable capacity	Survives	Typical use
RAID 1 (mirror)	50%	One disk	Hypervisor boot and small hosts
RAID 10	50%	One per mirror	High-IOPS guest storage
RAID 5 / RAIDZ1	n-1 disks	One disk	Capacity-oriented, write penalty
RAID 6 / RAIDZ2	n-2 disks	Two disks	Large arrays, long rebuilds

RAID is availability, not backup: a mirror faithfully replicates the delete you did not mean to run. Both copies are equally gone.

Network Interfaces and Traffic Separation

Traffic types to separate

- **Management** — the web UI, SSH, and cluster communication
- **Guest** — virtual machine production traffic, usually VLAN-tagged
- **Storage** — iSCSI and NFS to the TrueNAS nodes
- **Backup** — bulk transfer, scheduled off-hours

Why it matters

- A backup job can saturate a link and stall storage I/O for every guest on the host
- Management traffic on a guest network exposes the hypervisor to anything a guest can reach
- Storage links want jumbo frames (MTU 9000); guest links usually do not

Real-world constraint: when you have fewer NICs than traffic types, separate management and storage first, then VLAN the rest.

Pre-Installation Checklist

Complete this before booting the installer — every item costs minutes now and hours later.

- Confirm CPU virtualization extensions are enabled in UEFI, not merely present on the CPU.
- Apply current firmware to the system ROM, storage controller, and network adapters.
- Record disk count, size, and controller mode — Proxmox wants disks presented, not hidden behind hardware RAID.
- Reserve and record the management IP address, gateway, DNS server, and fully qualified hostname.
- Verify iLO5 access so you can reach the console when the network configuration is wrong.
- Confirm the ISO checksum before you write it to media.



The last item is the one people skip and the only one that produces a truly baffling installer failure.

LESSON 2

Installing Proxmox VE



Installation Media and Boot

Proxmox VE ships as a single bootable ISO with a Debian installer underneath.

- Download the ISO from proxmox.com and verify the published SHA-256 checksum.
- **iLO5 virtual media** mounts the ISO over the network — no physical trip to the rack.
- Boot the host and select the UEFI entry for the virtual optical device.
- The installer is a live environment: nothing is written to disk until you confirm the summary screen.



Out-of-band access

iLO5 gives you keyboard, video, and mouse independent of the operating system. When you misconfigure the management NIC — and you will — this is the only way back in.

LIVE DEMO

Mount the Proxmox ISO through iLO5 virtual media and boot one POD host.

Installer Modes

The boot menu offers three ways to run the same installation:

Mode	What it is	When to use it
Graphical	Full GUI installer with mouse support	Default choice on supported hardware
Terminal UI	Text-mode installer, identical prompts	Unsupported graphics or a slow remote console
Debug	Drops to a shell at defined stages	Diagnosing hardware the installer cannot see

Debug mode exits to a shell twice — before and after installation. Type exit to continue past each stop.

Choosing the Root Filesystem

The filesystem decision determines which features are available for the life of the host:

Filesystem	Strengths	Costs	Course use
ext4	Simple, predictable, low memory	No snapshots or checksums	Not used
ZFS	Snapshots, compression, checksums, RAID	Significant RAM appetite	Course default
BTRFS	Snapshots and subvolumes	Still marked technology preview	Awareness only

ZFS performs its own RAID. Present raw disks to it — never layer ZFS on top of a hardware RAID volume.

Configuring ZFS RAID1

Select ZFS RAID1 in the installer and confirm the disk selection and options:

- A **mirror** writes identical data to both disks; either one can fail and the host stays up.
- `ashift` sets the sector size exponent — `12` for 4K-sector drives, which is nearly all of them.
- **compression** defaults to `lz4` — leave it on, it usually costs less CPU than the I/O it saves.
- Usable capacity is half the raw total. Plan for it rather than discovering it after install.



One chance only

The RAID level and filesystem cannot be changed after installation. Changing your mind means a reinstall — which is cheap in the POD and expensive in production.

Management Network and Credentials

The final screens set the identity the node will carry for the rest of the course:

```
Hostname (FQDN) ... xxPOD01-Host1.mstclab.local
IP address ..... 10.xx.3.yy/24
Gateway ..... 10.xx.3.1
DNS server ..... 10.xx.2.254
Root password .... P@ssword01      (lab convention only)
```

- Replace **xx** with your site prefix — **22** for Wisconsin Rapids, **31** for Stevens Point.
- The hostname must be fully qualified and must resolve in DNS before you build a cluster.
- The email address is used for system notifications; a placeholder is fine in the lab.

First Login and Verification

A successful installation is one you can prove, not one that merely finished:

```
pveversion -v          # confirm the installed version and kernel
lscpu | grep -i virt   # confirm VT-x / AMD-V is exposed to the OS
zpool status rpool     # confirm the mirror is ONLINE with no errors
ip -br addr            # confirm vmbr0 holds the management address
```

- Browse to `https://xxPOD01-Host1.mstclab.local:8006` and accept the self-signed certificate.
- Log in as `root` with the **Linux PAM** realm — not the Proxmox VE authentication realm.
- The subscription notice on login is expected on the no-subscription repository.

LIVE DEMO

Run all four verification commands and read the zpool output aloud.

Post-Installation Tasks

A freshly installed host is not yet a host you would trust:

- Switch from the enterprise repository to the **no-subscription** repository, then update and reboot.
- Set the time zone and confirm **NTP** synchronization — clock skew breaks clustering and certificates.
- Record the installed version, disk layout, and network settings in your build documentation.
- Take a baseline snapshot of the configuration before you add a single virtual machine.



Repository choice

The enterprise repository requires a paid subscription and returns an authentication error without one. The no-subscription repository carries the same packages on a slower release cadence.

LIVE DEMO

Show the 401 from the enterprise repo, then switch repositories and update.

LESSON 3

Proxmox VE Architecture



Built on Debian

Proxmox VE is a management layer over a standard Debian Linux system.

- Broad hardware support arrives free with the **Debian kernel** and its driver set.
- Standard Linux tooling works — `systemctl`, `journalctl`, `ip`, and `ssh` behave exactly as expected.
- Proxmox services layer on top: `pveproxy` serves the web UI, `pvedaemon` executes API calls.
- Configuration lives in `/etc/pve`, a clustered filesystem replicated across every node.



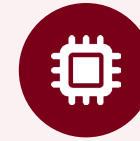
Discipline

Debian underneath is a convenience, not an invitation. Installing unrelated packages on a hypervisor adds attack surface and update risk to the one host every guest depends on.

KVM — Full Virtualization

KVM is a kernel module that makes Linux itself the hypervisor.

- Guest instructions execute **directly on the physical core** using VT-x or AMD-V — not emulated.
- **QEMU** provides device emulation around KVM: disks, NICs, and the virtual chipset.
- **VirtIO** drivers replace emulated devices with paravirtualized ones for near-native I/O.
- Each guest is an ordinary Linux process, so it is scheduled, limited, and monitored like one.



Install the VirtIO drivers

Windows guests do not ship with VirtIO drivers. Without them the guest falls back to emulated hardware and loses a large fraction of its disk and network throughput.

KVM and LXC Compared

KVM virtual machines

- Each guest runs its own complete kernel
- Any operating system — Windows, BSD, Linux
- Strong isolation enforced by the hardware
- Roughly 2–5% overhead; boots in tens of seconds

LXC containers

- Share the host kernel through namespaces and cgroups
- Linux workloads only, matching the host kernel
- Process-level isolation; a kernel flaw crosses the boundary
- Under 1% overhead; boots in about a second

Choosing between them: if the workload needs its own kernel, a different OS, or a hard security boundary, use a VM. Otherwise, a container is cheaper.

The Virtualization Stack

Four layers, each with a distinct job and a distinct performance signature:

Layer	Role	Technology	Overhead
Debian base	Host operating system	Linux kernel	Minimal
KVM + QEMU	Full virtualization	Hardware-assisted	2–5%
LXC	Container virtualization	Namespaces and cgroups	Under 1%
ZFS	Storage management	Copy-on-write filesystem	Varies with features

Overhead figures are typical for general workloads; heavy I/O and nested virtualization both push them higher.

Storage and Network Abstraction

Proxmox presents one consistent interface over very different physical resources.

- **Storage plugins** abstract local disks, NFS exports, iSCSI targets, and Ceph behind one API.
- A **Linux bridge** — normally `vmbr0` — is the virtual switch that guests attach to.
- **VLAN tagging** on a bridge carries many isolated networks over one physical link.
- Which backend a guest uses determines whether it can migrate, snapshot, or fail over.



Coming up

Local storage cannot support live migration between nodes. That constraint is the reason the TrueNAS shared storage arrives in the storage and networking units.

Type 1 and Type 2 Hypervisors

Type 1 — bare metal

- Installs directly on hardware; no host OS beneath it
- Direct access to CPU extensions, MMU, and controllers
- Small code base means a small attack surface
- Supports live migration, HA clustering, and SR-IOV

Type 2 — hosted

- Runs as an application on a general-purpose OS
- Hardware access mediated by the host operating system
- Inherits the entire host OS attack surface
- Excellent for a workstation; unsuited to production

Measured difference: Type 1 hypervisors typically reach 95–98% of native performance against roughly 85–90% for Type 2, and the gap widens under sustained load.

Where Proxmox VE Sits

Platform selection is a licensing and staffing decision at least as much as a technical one:

Platform	Licensing model	Management	Strongest fit
Proxmox VE	Open source, optional support	Built into every node	Mixed workloads, cost-sensitive
Microsoft Hyper-V	Included with Windows Server	Windows Admin Center	Windows-centric shops
XCP-ng	Open source, optional support	Xen Orchestra	Xen-based environments
oVirt / RHV	Subscription	Separate management host	Red Hat ecosystems

This course teaches Proxmox VE and Hyper-V because they are the two platforms most Wisconsin employers are hiring for now.



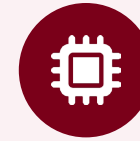
LESSON 4

Capacity Planning and Best Practices

CPU Oversubscription and NUMA

Virtual CPUs are scheduled time on physical cores, not dedicated hardware.

- Ratios of **2:1 to 4:1** vCPU to physical core are typical; compute-bound workloads need less.
- **CPU ready time** — time a guest was runnable but waiting — is the real contention signal.
- Sustained ready time above **5%** means the host is oversubscribed regardless of utilization.
- Oversized guests schedule worse than right-sized ones. Start small and grow on evidence.



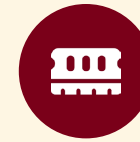
NUMA locality

On a multi-socket host, memory attached to the other socket is measurably slower to reach. A guest that fits inside one NUMA node performs better than one straddling two.

Memory Overcommitment

Memory can be promised more generously than it can be delivered — within limits.

- **Ballooning** lets the host reclaim unused memory from idle guests and lend it where it is needed.
- **Transparent page sharing** deduplicates identical pages, often saving 20–40% across similar guests.
- Ratios above **1.5:1** allocated to physical deserve a documented justification.
- Reservations guarantee memory for critical guests; limits stop one guest from starving the rest.



Where it breaks

When overcommitted memory is actually claimed, the host swaps to disk. Performance does not degrade gracefully — it collapses, and it collapses for every guest at once.

Storage Performance Planning

Guests consolidate their I/O onto shared spindles — plan for the aggregate, not the average.

- **IOPS** and **latency** matter more than capacity; a full-looking datastore is rarely the real problem.
- Sustained latency above **20 ms** is a bottleneck a user can feel.
- Random I/O from many guests defeats spinning media — this is why flash dominates virtualization.
- Backup windows compete with production I/O. Schedule them, do not hope.



Lab standards

Storage and SAN networks in the POD run **MTU 9000** end to end. TrueNAS zvols use a **4k** block size for Proxmox compatibility.

Resource Monitoring Thresholds

Know the metric that reveals contention for each resource, and the value that means act:

Resource	Typical ratio	Watch this metric	Trouble signal
CPU	2:1 to 4:1	Ready time, utilization	Ready time above 5%
Memory	1.2:1 to 1.5:1	Active memory, ballooning	Host swapping to disk
Storage	Workload dependent	IOPS, latency, queue depth	Latency above 20 ms
Network	Application dependent	Utilization, packet loss	Sustained use above 80%

Baseline these on a quiet host before you need them — a threshold without a baseline is a guess.

Installation Best Practices

What separates a build you can support from one you merely finished:

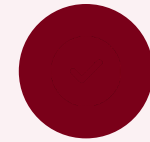
- Update firmware **before** installing — outdated system ROM causes failures that look like software bugs.
- Check the hardware compatibility list before procurement, not after the boxes arrive.
- Establish a **performance baseline** on the new host while it is still idle and uncontended.
- Keep configuration identical across nodes that will cluster; drift surfaces as intermittent HA failures.
- Verify power and cooling headroom for the fully loaded host, not the idle one.

Cluster rule: nodes that will belong to one cluster should be built from one documented procedure, not from memory four times.

Documentation and Change Control

The build record is part of the build — not an afterthought.

- Capture hardware, firmware levels, storage layout, network settings, and account decisions.
- Diagram the network topology; connectivity problems are solved on paper before they are solved in a shell.
- Record **why** each choice was made — the reasoning outlives the person who made it.
- Test the recovery procedure at build time, while failure is still a rehearsal.



Regulated environments

HIPAA, PCI DSS, and SOX all impose specific logging, access control, and documentation requirements. Build them in at install time — retrofitting compliance is far more expensive.

Security Focus: The Management Plane

Every security decision in this unit points at the same asset — access to the hypervisor itself.



SECURITY FOCUS

Compromise of a hypervisor is compromise of every guest it hosts. An attacker on the management plane does not need to defeat any guest operating system — they can copy disks, attach consoles, or clone virtual machines wholesale.

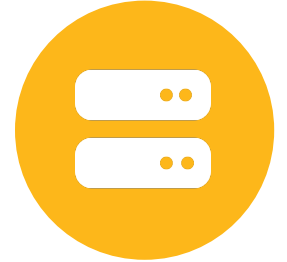
Isolate management on its own VLAN or physical interface, restrict the web UI and SSH to administrative networks, replace lab-convention passwords before anything real runs, and enable audit logging from day one.

Why it matters: A guest with perfect internal security is still fully exposed if the hypervisor beneath it is not. The management plane is the shortest path an attacker has to every workload you run, which makes protecting it the highest-value control in this unit.

Hands-On Lab: Installing Proxmox VE

POD lab · your assigned HP DL20 Gen 10 host · console access through iLO5

- Complete the pre-installation checklist and record the results before booting the ISO.
- Mount the Proxmox VE ISO through iLO5 virtual media and boot the installer.
- Install with ZFS RAID1, using your assigned hostname, IP address, gateway, and DNS server.
- Switch to the no-subscription repository, update the host, and reboot.
- Verify the installation with pveversion, zpool status, and a successful web UI login.
- Submit a build document capturing every configuration decision you made.



Graded on a reachable, verified host and the build document — not on screenshots of the installer.

Key Takeaways

-  Storage layout, network identity, and CPU features are decided before install and are effectively permanent.
-  ZFS RAID1 protects the hypervisor; verification, not a finished progress bar, proves the install.
-  Proxmox VE is Debian plus KVM for full virtual machines and LXC for containers.
-  Oversubscribe deliberately: watch CPU ready time, memory swapping, and storage latency.
-  Whoever reaches the management plane owns every guest on the host.

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Questions?

Unit 2 · Hypervisor Installation and Architecture